
Lecture 04

Files, JSON, Exceptions and LINQ

Basics of Internet Technologies 2

Making console data survive restart

Learning Objectives

- Explain memory vs persistent storage.
- Use file paths and create directories.
- Read and write text files.
- Explain JSON as structured text.
- Use `System.Text.Json` for serialization and deserialization.
- Handle runtime problems with `try`, `catch`, and `finally`.
- Use LINQ `Where`, `Select`, `FirstOrDefault`, `Any`, and `OrderBy`.
- Add automatic load/save to a console application.

Connection to Lecture 03

- Lecture 03 made the application work with many items.
- The data lived in `List<T>` while the program was running.
- When the program stopped, the list disappeared.
- Today we keep data after restart by saving it to a file.

Lesson Timing Plan

- 10 min - problem: memory disappears.
- 15 min - paths, directories, text files.
- 20 min - JSON structure and `System.Text.Json`.
- 20 min - load/save demo with `List<Dictionary<string,string>>`.
- 20 min - exceptions and try/catch/finally.
- 20 min - LINQ minimum for searching and filtering.
- 15 min - live coding persistence into the expense app.
- 10 min - Milestone 01 and exercises.

Problem: Data Disappears

```
List<string> names = new List<string>();  
names.Add("Coffee");  
  
// Program stops.  
// Memory is gone.
```

- Memory is temporary.
- Variables and lists live while the process runs.
- Persistent storage keeps data outside the running program.

Memory vs Persistent Storage

Memory

- Fast
- Temporary
- Cleared when the program ends
- Variables and lists

Persistent Storage

- Slower
- Survives restart
- Stored on disk or database
- Files, JSON, SQL later

Files in This Lecture

- A file is data stored on disk.
- A text file stores characters.
- A JSON file stores structured text.
- Today we use files before databases so students see the persistence problem clearly.

Paths

```
string dataDirectory = Path.Combine(  
    Directory.GetCurrentDirectory(),  
    "data");  
  
string filePath = Path.Combine(dataDirectory, "expenses.json");
```

- A path tells the program where a file or folder is.
- Path.Combine builds paths safely for the operating system.
- Directory.GetCurrentDirectory returns where the program is running from.

Creating a Directory

```
Directory.CreateDirectory(dataDirectory);  
  
Console.WriteLine(Directory.Exists(dataDirectory));
```

- CreateDirectory creates the folder if it does not exist.
- It is safe to call even when the folder already exists.
- Directory.Exists checks whether a directory is present.

Writing a Text File

```
File.WriteAllText(filePath, "Coffee\nTea\nLunch");
```

- WriteAllText creates or overwrites a text file.
- The second argument is the text content.
- Overwriting is simple, but later we will think about data safety.

Reading a Text File

```
string text = File.ReadAllText(filePath);  
Console.WriteLine(text);
```

- ReadAllText reads the whole file as one string.
- This is easy for small teaching examples.
- Large files may need other approaches later.

Reading Lines

```
string[] lines = File.ReadAllLines(filePath);

foreach (string line in lines)
{
    Console.WriteLine(line);
}
```

- ReadAllLines gives an array of lines.
- This connects file reading to arrays and foreach.
- Line-based files are simple but not structured enough for our app.

Why JSON?

- A list of lines loses meaning quickly.
- JSON can represent structured data with names and values.
- JSON is common in web APIs and configuration files.
- Learning JSON now prepares students for HTTP and APIs later.

JSON Object

```
{  
  "name": "Coffee",  
  "amount": "4.50",  
  "category": "Food"  
}
```

- An object is wrapped in braces.
- Each property has a name and a value.
- JSON property names are usually written in double quotes.

JSON Array

```
[  
  { "name": "Coffee", "amount": "4.50" },  
  { "name": "Bus", "amount": "2.80" }  
]
```

- An array is wrapped in square brackets.
- A JSON array can contain several objects.
- This matches our list of expenses.

Temporary Structure Before Classes

```
List<Dictionary<string, string>> expenses =  
    new List<Dictionary<string, string>>();
```

- Classes are introduced in Lecture 05.
- For today, a dictionary can represent one simple JSON-like object.
- This keeps the focus on files and JSON, not object modeling.

Creating One Dictionary Item

```
Dictionary<string, string> expense = new Dictionary<string, string>();  
expense["name"] = "Coffee";  
expense["amount"] = "4.50";  
expense["category"] = "Food";
```

- The dictionary key is the property name.
- The dictionary value is the stored text value.
- This is not final architecture. It is a bridge toward classes.

System.Text.Json

- System.Text.Json is the built-in .NET JSON library.
- Serialization means C# value to JSON text.
- Deserialization means JSON text back to C# value.
- The same idea appears later in Web API request and response bodies.

Serialization

```
JsonSerializerOptions options = new JsonSerializerOptions();  
options.WriteIndented = true;  
  
string json = JsonSerializer.Serialize(expenses, options);
```

- Serialize converts C# data to JSON text.
- WriteIndented makes the file easier for humans to read.
- Pretty JSON is helpful during teaching and debugging.

Saving JSON

```
Directory.CreateDirectory(dataDirectory);  
File.WriteAllText(jsonPath, json);
```

- Create the directory before writing into it.
- WriteAllText stores the JSON string in a file.
- After this, data can survive application restart.

Deserialization

```
string json = File.ReadAllText(jsonFilePath);  
List<Dictionary<string, string>> items =  
    JsonSerializer.Deserialize<List<Dictionary<string, string>>>(json);
```

- Deserialize converts JSON text back to C# data.
- The type must match the expected JSON shape.
- Invalid JSON can cause a `JsonException`.

Automatic Load on Startup

```
List<Dictionary<string, string>> expenses =  
    LoadExpenses(dataFilePath);
```

- The program loads saved data before showing the menu.
- If the file does not exist, start with an empty list.
- This keeps the first run simple and friendly.

Automatic Save After Changes

```
AddExpense(expenses);  
SaveExpenses(dataFilePath, expenses);
```

- Save after adding or deleting data.
- This reduces the chance of losing work.
- Later architecture will move this responsibility away from Program.cs.

Exceptions

- An exception represents a runtime problem.
- Examples: invalid number, missing file, invalid JSON, no permission.
- Without handling, an exception can stop the program.
- With handling, the program can show a useful message or recover.

try and catch

```
try
{
    string text = File.ReadAllText(path);
}
catch (FileNotFoundException ex)
{
    Console.WriteLine("File not found: " + ex.FileName);
}
```

- Code that may fail goes inside try.
- catch handles a specific exception type.
- The exception object contains information about the problem.

Handling Invalid JSON

```
catch (JsonException)
{
    Console.WriteLine("The JSON file is invalid.");
    return new List<Dictionary<string, string>>();
}
```

- Invalid data should not destroy the lesson demo.
- The app can start with an empty list after warning the user.
- In production, we would be more careful with backups and logging.

finally

```
try
{
    Console.WriteLine("Trying work");
}
catch (IOException ex)
{
    Console.WriteLine(ex.Message);
}
finally
{
    Console.WriteLine("Always runs");
}
```

- finally runs whether try succeeded or failed.
- It is useful for cleanup.
- Many modern APIs handle cleanup with using, but finally is important conceptually.

Do Not Catch Everything Silently

- A catch block should not hide every problem without explanation.
- Show useful beginner messages.
- Catch specific exception types when possible.
- Silent failure makes debugging confusing.

LINQ: Why It Helps

- LINQ means Language Integrated Query.
- It lets us ask questions about collections clearly.
- Today we use a small practical subset only.
- LINQ becomes very important again with Entity Framework Core.

Where

```
List<Dictionary<string, string>> foodExpenses = expenses
    .Where(expense => expense["category"] == "Food")
    .ToList();
```

- Where filters a collection.
- The expression decides which items stay.
- ToList turns the query result into a list.

Select

```
List<string> names = expenses
    .Select(expense => expense["name"])
    .ToList();
```

- Select transforms each item into another value.
- Here each expense becomes only its name.
- This is useful for projections and display data.

FirstOrDefault

```
Dictionary<string, string> firstFood = expenses  
    .FirstOrDefault(expense => expense["category"] == "Food");
```

- FirstOrDefault returns the first matching item.
- If nothing matches, it returns null for reference types.
- Check for null before using the result.

Any

```
bool hasFood = expenses
    .Any(expense => expense["category"] == "Food");
```

- Any answers a yes/no question.
- It is useful for checks such as exists, has items, has category.
- The result is bool.

OrderBy

```
List<Dictionary<string, string>> ordered = expenses
    .OrderBy(expense => expense["name"])
    .ToList();
```

- OrderBy sorts items by a selected value.
- It does not change the original list unless you store the result.
- Sorting makes list output easier to scan.

Live Demo Menu

1. Add expense
2. List expenses
3. Search by name
4. Filter by category
5. Delete expense
6. Show summary
7. Save
8. Exit

- The demo loads JSON on startup.
- Add and delete automatically save JSON.
- Search, filter, order, and summary use LINQ.

Demo Data File

```
data/expenses.json
```

- The file is placed in a data folder under the current working directory.
- The teacher can open the JSON file and show the saved structure.
- Editing JSON manually is possible, but invalid JSON should be handled.

Student Project Increment

- Add automatic load when the application starts.
- Add save after meaningful changes.
- Store data in a JSON file.
- Use try/catch for missing file, invalid JSON, and invalid input.
- Add searching or filtering with LINQ.
- Keep the project single-user and console-based.
- After this lecture, save Milestone 01 - Basic Console.

Milestone 01 - Basic Console

- Console menu exists.
- Conditions and loops are used.
- Methods organize repeated work.
- Collections store multiple items.
- Basic CRUD-like behaviour exists: Add, List, Find, Delete.
- Data is saved to a JSON file.
- The student can explain what changed from Lecture 01 to Lecture 04.

What Not To Add Yet

- Do not add classes yet unless the teacher explicitly asks for preview work.
- Do not add interfaces or repositories yet.
- Do not add users or login yet.
- Do not add SQL or EF Core yet.
- Do not build a Web API yet.
- The goal is persistence and basic collection querying.

Common Mistakes Today

- Saving to a path without creating the folder first.
- Assuming the file always exists.
- Assuming JSON is always valid.
- Catching an exception but not telling the user what happened.
- Forgetting that deserialization can return null.
- Using LINQ before understanding the equivalent loop.
- Trying to make the JSON structure too advanced before classes.

Exercises

- Add a command that shows only expenses above a chosen amount.
- Add a command that prints all unique categories.
- Make the app show where the JSON file is stored.
- Intentionally break the JSON file and observe the error handling.
- Use `OrderBy` to sort by category or name.
- Write a short Milestone 01 explanation for your project.

Review Questions

- What is the difference between memory and persistent storage?
- Why do we create the data directory before saving?
- What is JSON?
- What is serialization?
- What is deserialization?
- What is an exception?
- Why should catch blocks be specific?
- What does Where do?
- What does Select do?
- When should the student save Milestone 01?

Summary

- Files let data survive program restart.
- JSON stores structured text and prepares students for APIs.
- `System.Text.Json` converts C# data to JSON and back.
- Exceptions let programs handle runtime problems.
- LINQ gives readable collection queries.
- Lecture 04 completes the first Basic Console milestone.